

Data Structure and Algorithm (PMSCS 623P)

Lecture 15 Shortest Path Problem

Shortest Path Problems

Modeling problems as graph problems:

Road map is a weighted graph:

vertices = cities

edges = road segments between cities

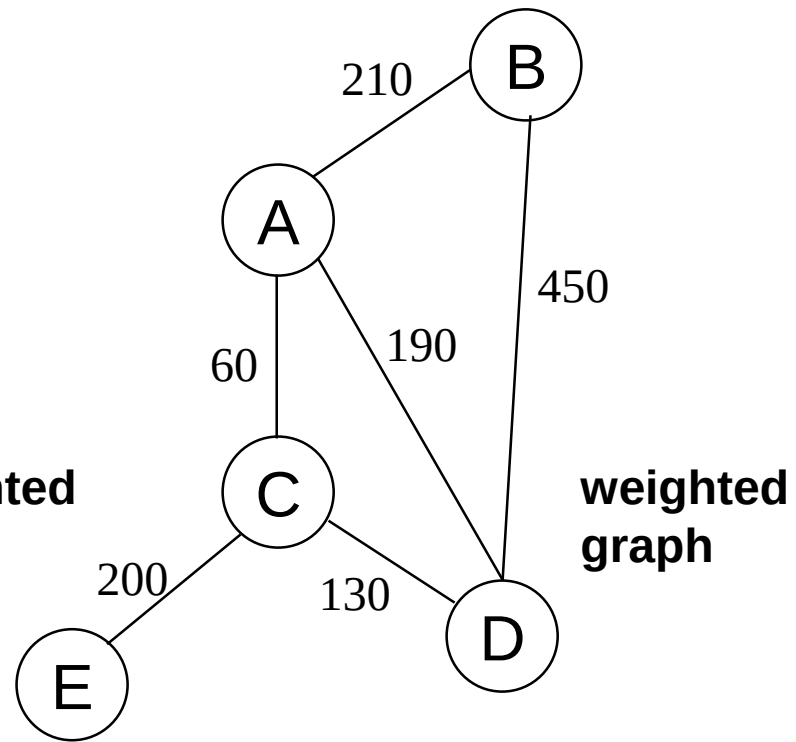
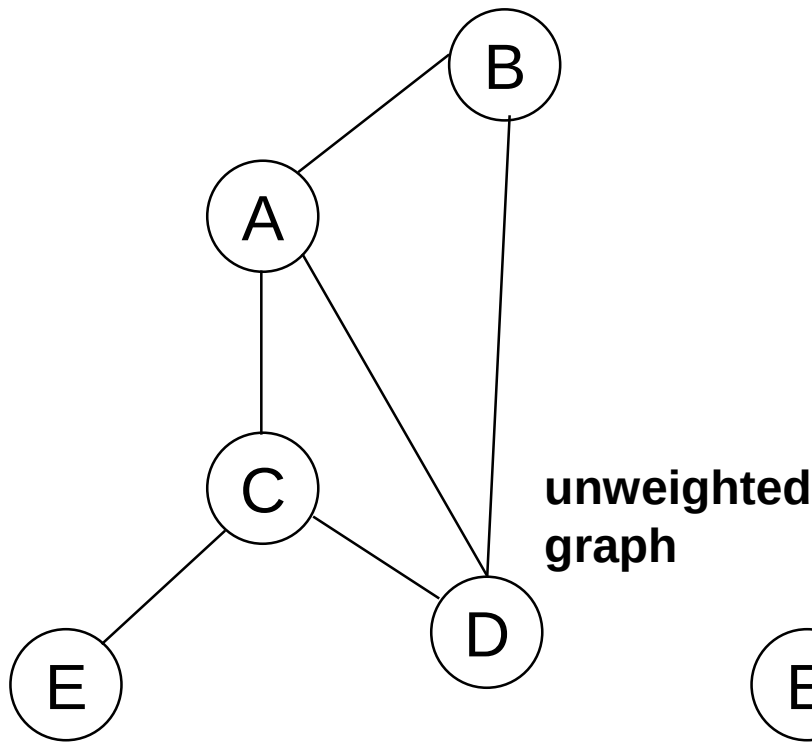
edge weights = road distances

Goal: find a shortest path between two vertices (cities)

Shortest Path Problems

What is shortest path ?

- shortest length between two vertices for an unweighted graph:
- smallest **cost** between two vertices for a weighted graph:

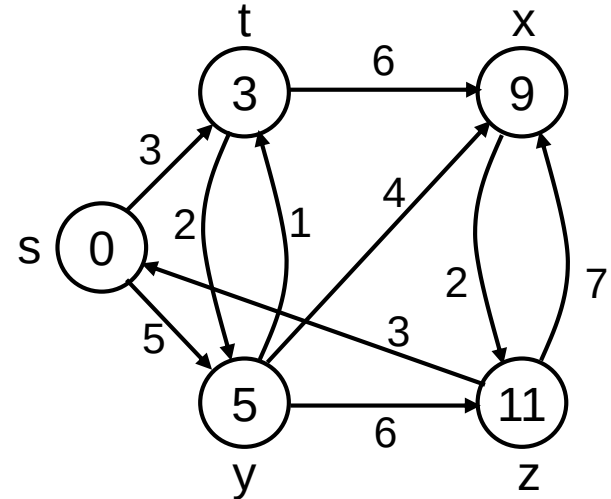


Shortest Path Problems

- **Input:**

- Directed graph $G = (V, E)$
- Weight function $w : E \rightarrow \mathbf{R}$

- **Weight of path** $p = \langle v_0, v_1, \dots, v_k \rangle$
$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$$



- **Shortest-path weight** from u to v :

$$\delta(u, v) = \min \begin{cases} w(p) : u \rightsquigarrow^p v & \text{if there exists a path from } u \text{ to } v \\ \infty & \text{otherwise} \end{cases}$$

Variants of Shortest Paths

Single-source shortest path

Given $G = (V, E)$, find a shortest path from a given source vertex s to each vertex $v \in V$

Single-destination shortest path

Find a shortest path to a given destination vertex t from each vertex v

Reverse the direction of each edge $\Rightarrow$ single-source

Single-pair shortest path

Find a shortest path from u to v for given vertices u and v

Solve the single-source problem

All-pairs shortest-paths

Find a shortest path from u to v for every pair of vertices u and v

Optimal Substructure of Shortest Paths

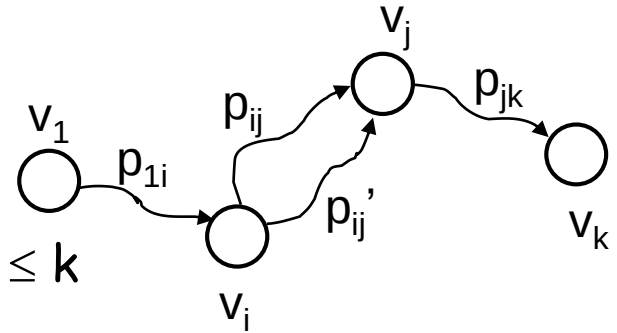
Given:

A weighted, directed graph $G = (V, E)$

A weight function $w: E \rightarrow \mathbf{R}$,

A shortest path $p = \langle v_1, v_2, \dots, v_k \rangle$ from v_1 to v_k

A subpath of p : $p_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$, with $1 \leq i \leq j \leq k$



Then: p_{ij} is a shortest path from v_i to v_j

Proof: Let $p = v_1 \xrightarrow{p_{1i}} v_i \xrightarrow{p_{1j}} v_j \xrightarrow{p_{jk}} v_k$

$$\Rightarrow w(p) = w(p_{1i}) + w(p_{1j}) + w(p_{jk})$$

Assume $\exists p'_{ij}$ from v_i to v_j with $w(p'_{ij}) < w(p_{1j})$

Adding $w(p_{1i}) + w(p_{jk})$ in both sides of this inequality:

$$\Rightarrow w(p') = w(p_{1i}) + w(p'_{ij}) + w(p_{jk}) < w(p_{1i}) + w(p_{1j}) + w(p_{jk}) = w(p)$$

So there is a path p' from v_1 to v_k which is shorter than the shortest path p between them;
but this contradicts our initial assumption that p is the shortest path.

Shortest-Path Idea

- Recall: $\delta(u, v) \equiv$ total weight/cost of the shortest path from u to v .
- All SSSP algorithms maintain a field $d[u]$ for every vertex u . $d[u]$ will be an estimate of $\delta(s, u)$. As the algorithm progresses, we will refine $d[u]$ until, at termination,

$$d[u] = \delta(s, u).$$

- Whenever we discover a new shortest path to u , we update $d[u]$.
In fact, $d[u]$ will always be an *overestimate* of $\delta(s, u)$:

$$d[u] \geq \delta(s, u)$$

- We'll use $\pi[u]$ to point to the parent (or predecessor) of u on the shortest path from s to u . We update $\pi[u]$ when we update $d[u]$.
- At the end, π will induce a tree, called **shortest path tree**.

Initialization

Alg.: INITIALIZE-SINGLE-SOURCE(V, s)

1. **for** each $v \in V$
2. **do** $d[v] \leftarrow \infty$
3. $\pi[v] \leftarrow \text{NIL}$
4. $d[s] \leftarrow 0$

All the shortest-paths algorithms start with INITIALIZE-SINGLE-SOURCE

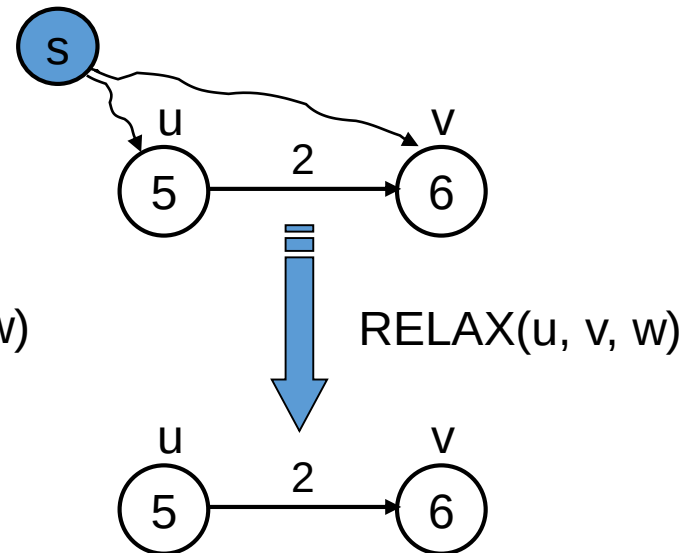
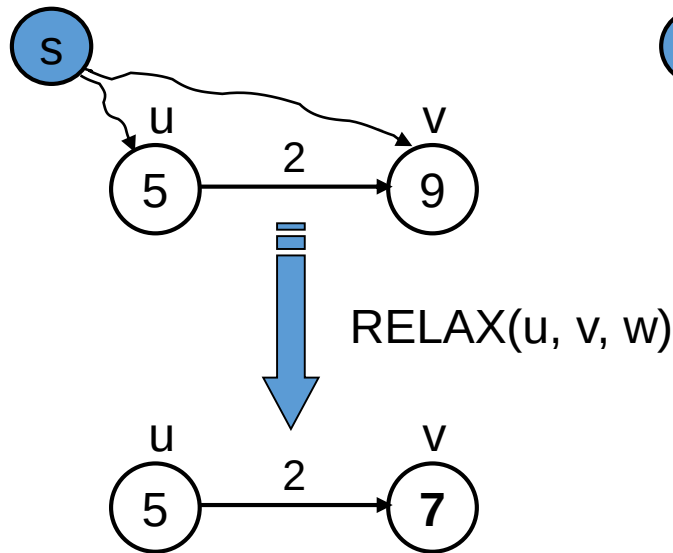
Relaxation

Relaxing an edge (u, v) = testing whether we can improve the shortest path to v found so far by going through u

If $d[v] > d[u] + w(u, v)$

we can improve the shortest path to v

$\Rightarrow$ update $d[v]$ and $\pi[v]$



After relaxation: $d[v] \leq d[u] + w(u, v)$

RELAX(u, v, w)

1. if $d[v] > d[u] + w(u, v)$
2. then $d[v] \leftarrow d[u] + w(u, v)$
3. $\pi[v] \leftarrow u$

All the single-source shortest-paths algorithms
start by calling INIT-SINGLE-SOURCE
then relax edges

The algorithms differ in the order and how many times they
relax each edge

Dijkstra's Algorithm

(pronounced "DIKE-stra")

Solves Single-source Shortest Path (SSSP) problem:

When there is no negative-weight edges: $w(u, v) > 0 \forall (u, v) \in E$

Maintains two sets of vertices:

K = vertices whose final shortest-path weights have already been determined

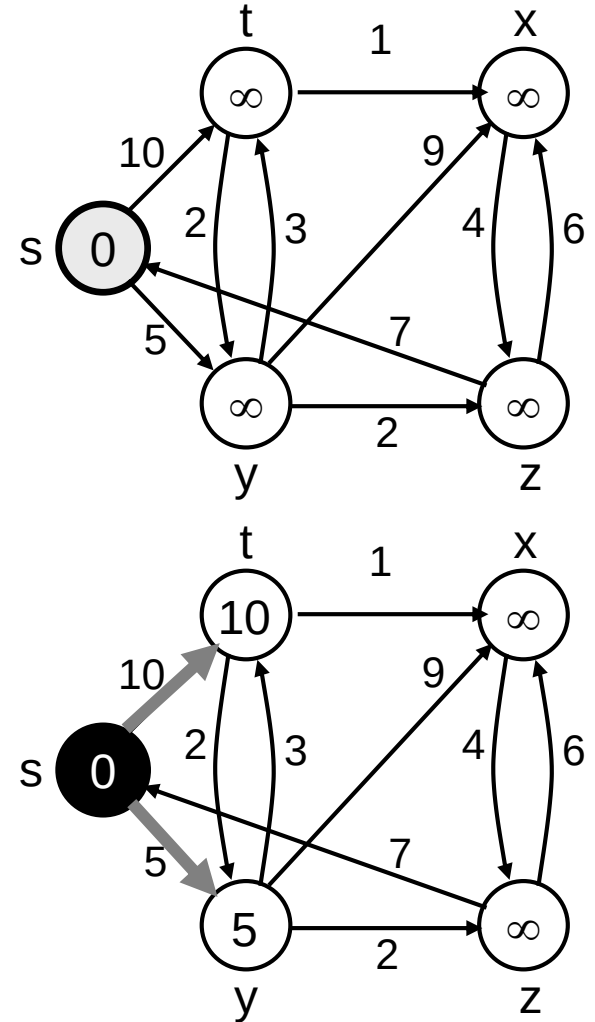
Q = vertices in $V - K$ (Q is a min-priority queue)

Keys in Q are estimates of shortest-path weights ($d[v]$)

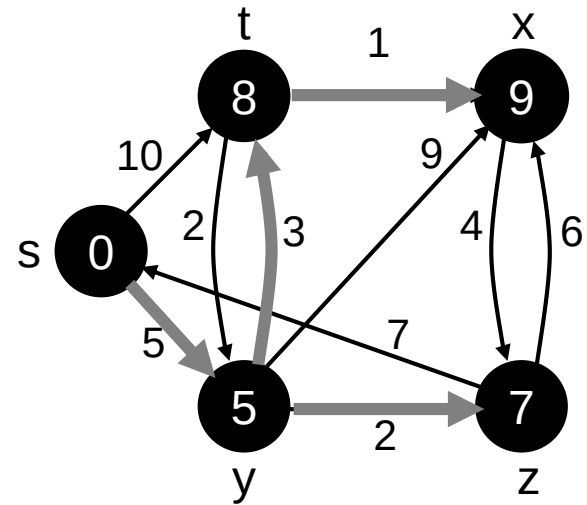
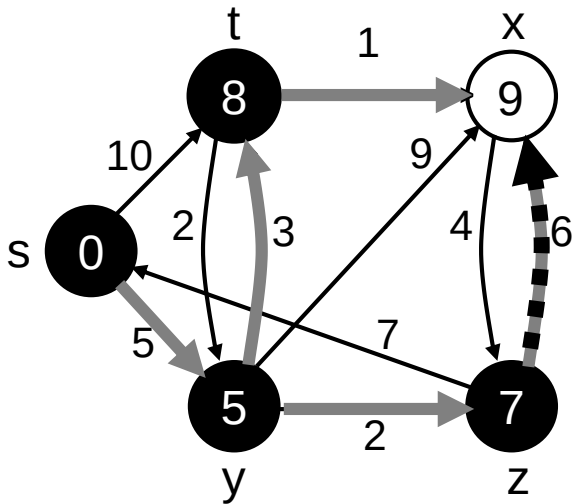
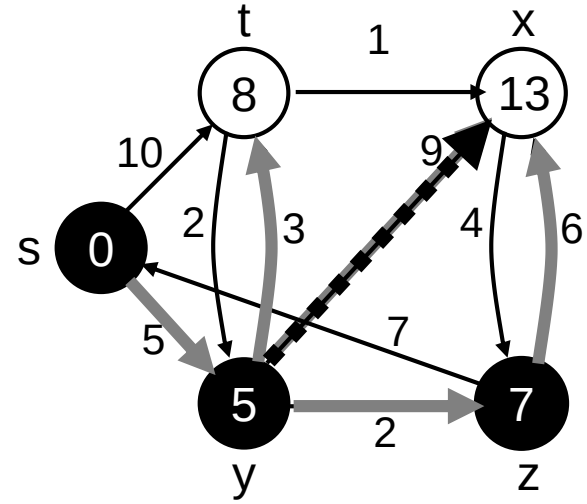
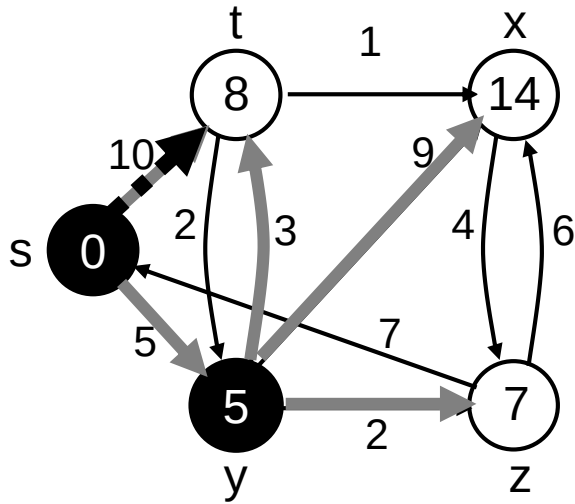
Repeatedly select a vertex $u \in V - K$, with the minimum shortest-path estimate $d[u]$ and RELAXs its incident edges.

Dijkstra (G, w, s)

1. INITIALIZE-SINGLE-SOURCE(V, s)
2. $K \leftarrow \emptyset$
3. $Q \leftarrow V[G]$
4. **while** $Q \neq \emptyset$
5. **do** $u \leftarrow \text{EXTRACT-MIN}(Q)$
6. $K \leftarrow K \cup \{u\}$
7. **for** each vertex $v \in \text{Adj}[u]$
8. **do** RELAX(u, v, w)



Example



Dijkstra's Pseudo Code

Graph G , weight function w , root s

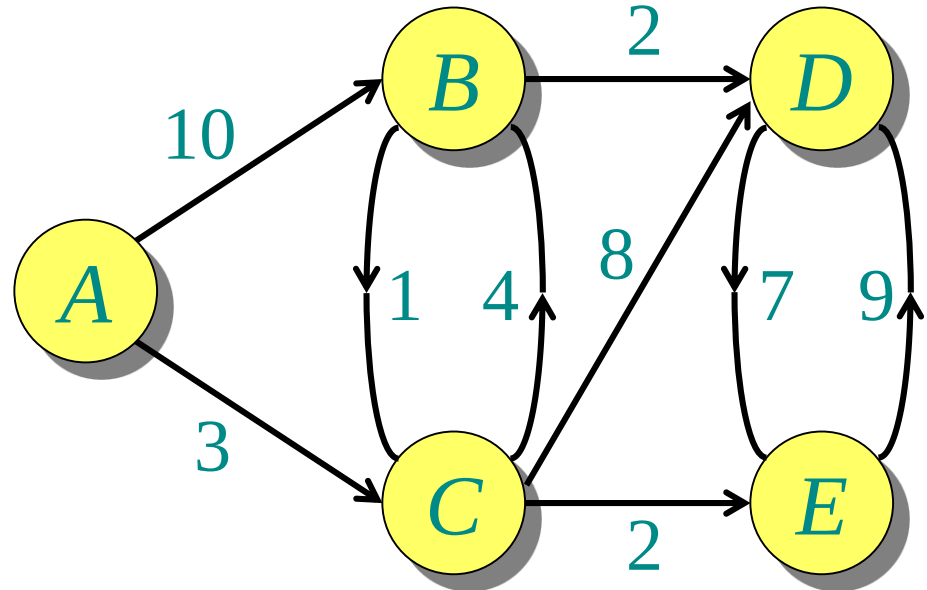
DIJKSTRA(G, w, s)

```
1 for each  $v \in V$ 
2     do  $d[v] \leftarrow \infty$ 
3  $d[s] \leftarrow 0$ 
4  $S \leftarrow \emptyset$   $\triangleright$  Set of discovered nodes
5  $Q \leftarrow V$ 
6 while  $Q \neq \emptyset$ 
7     do  $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
8          $S \leftarrow S \cup \{u\}$ 
9         for each  $v \in \text{Adj}[u]$ 
10             do if  $d[v] > d[u] + w(u, v)$ 
11                 then  $d[v] \leftarrow d[u] + w(u, v)$ 
```

relaxing
edges

Example of Dijkstra's algorithm

**Graph with
nonnegative
edge weights:**

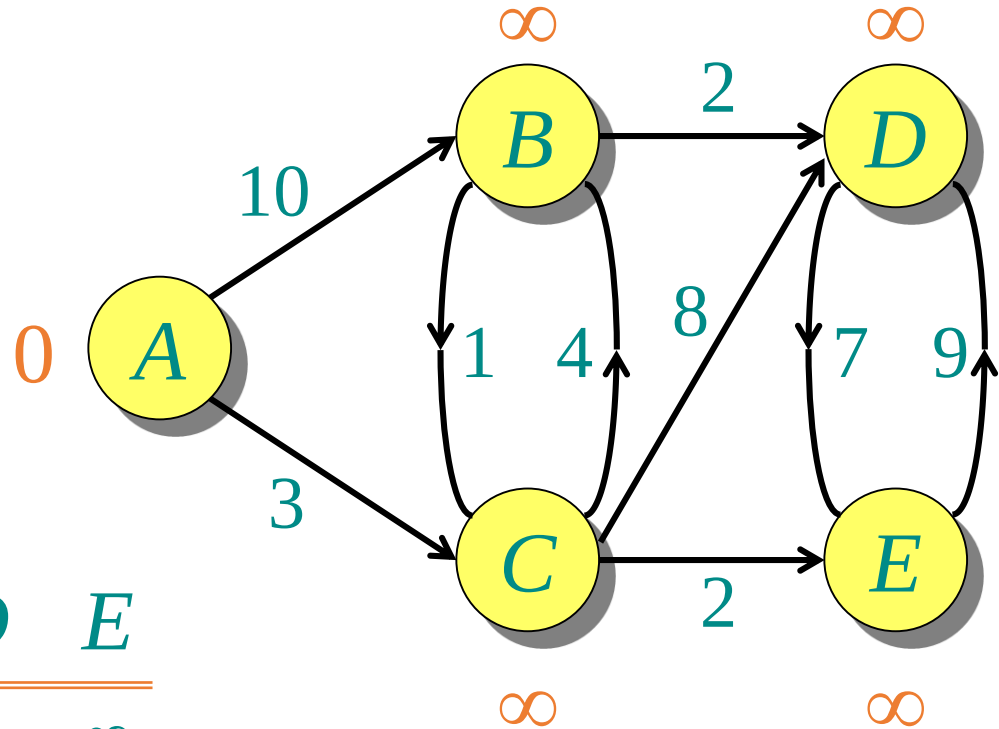


Example of Dijkstra's algorithm

Initialize:

$K: \{\}$

$Q:$	A	B	C	D	E
	0	∞	∞	∞	∞

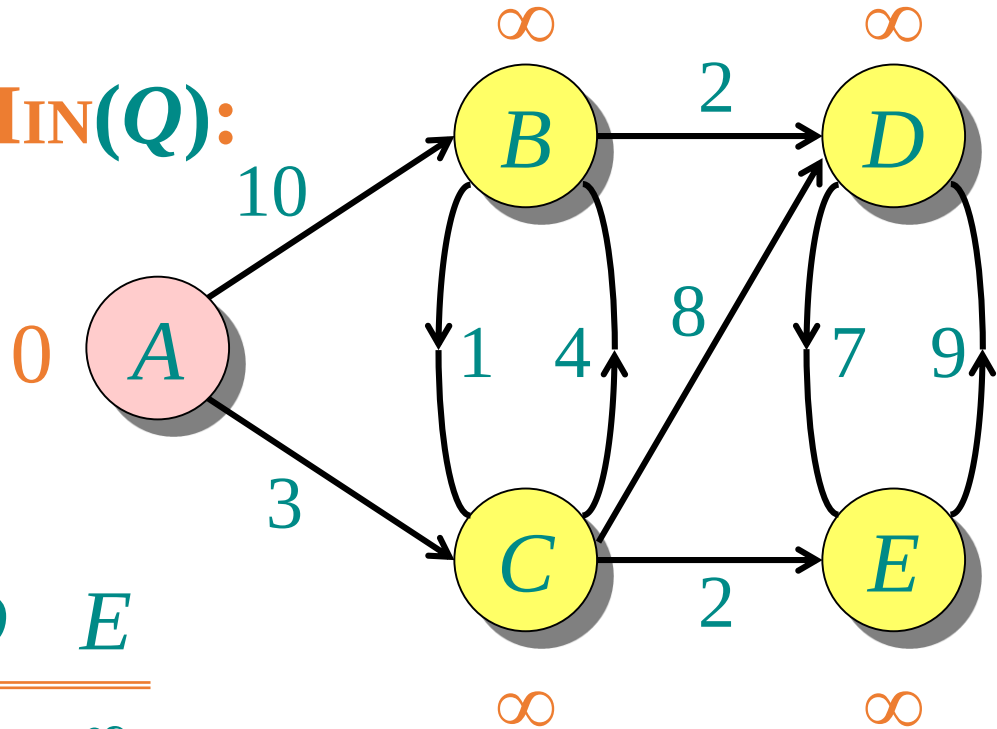


Example of Dijkstra's algorithm

“A” $\leftarrow$ **EXTRACT-MIN**(Q):

$K: \{A\}$

Q:	A	B	C	D	E
	0	∞	∞	∞	∞

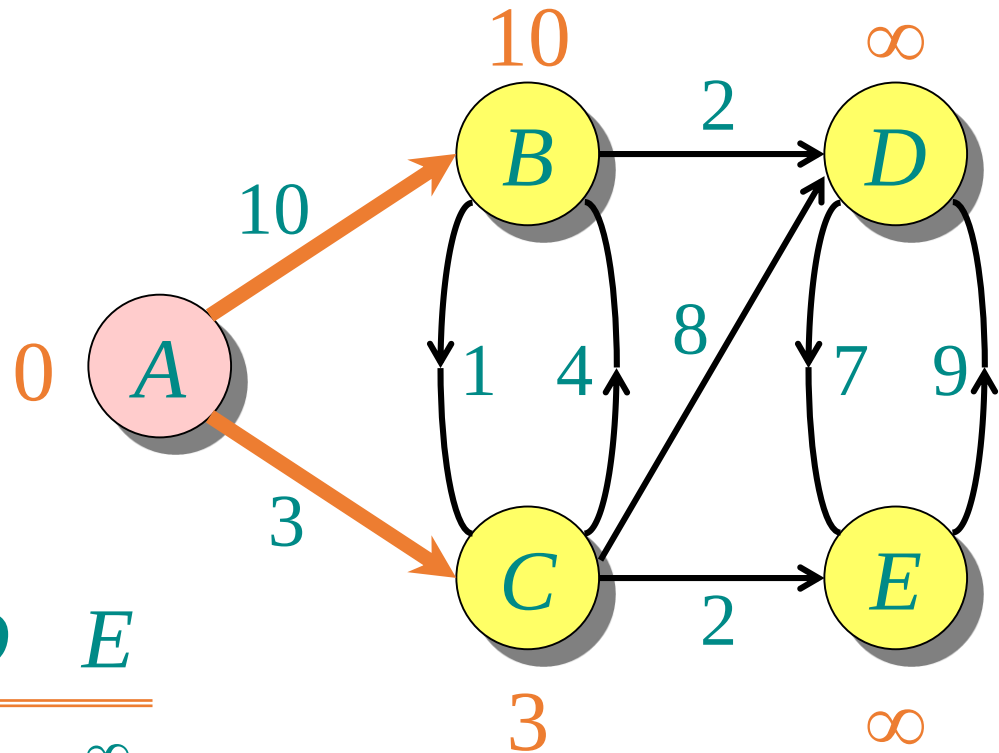


Example of Dijkstra's algorithm

Relax all edges
leaving A :

$K: \{A\}$

$Q:$	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	—	—

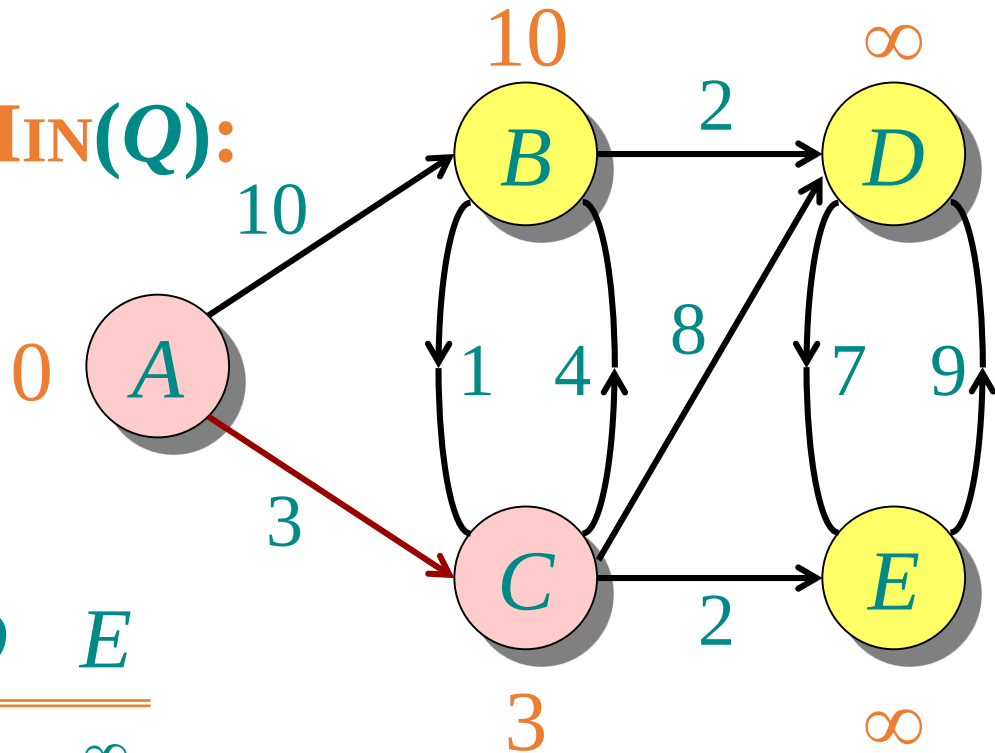


Example of Dijkstra's algorithm

“C” ← **EXTRACT-MIN**(Q):

$K: \{A, C\}$

Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	—	—

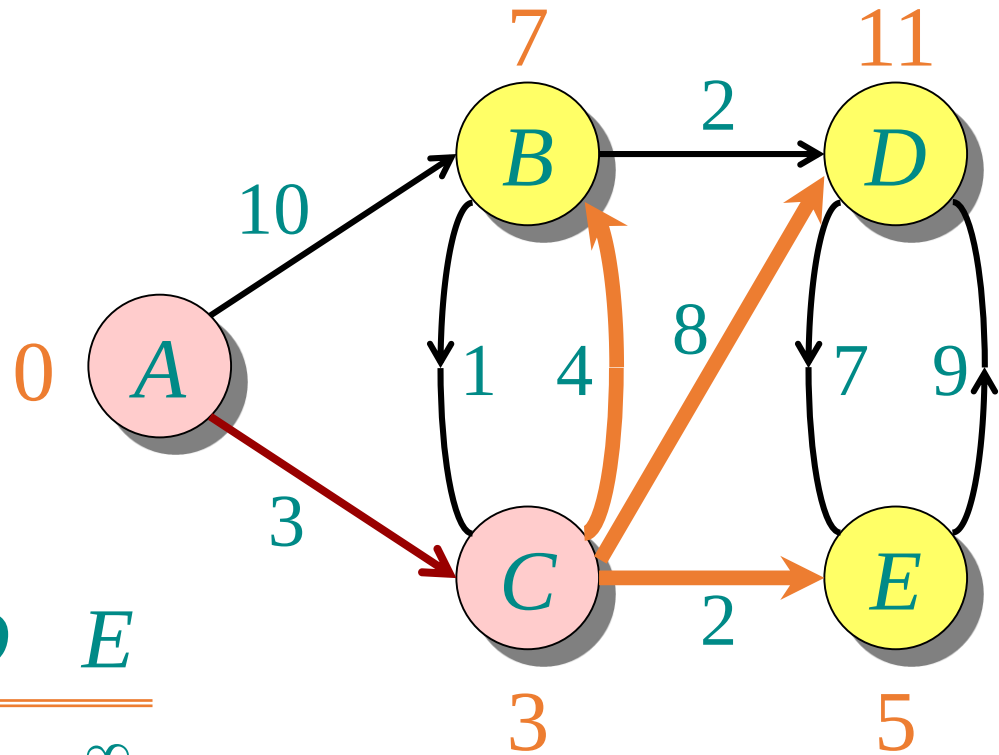


Example of Dijkstra's algorithm

Relax all edges
leaving **C**:

$K: \{A, C\}$

Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	—	—
		7		11	5

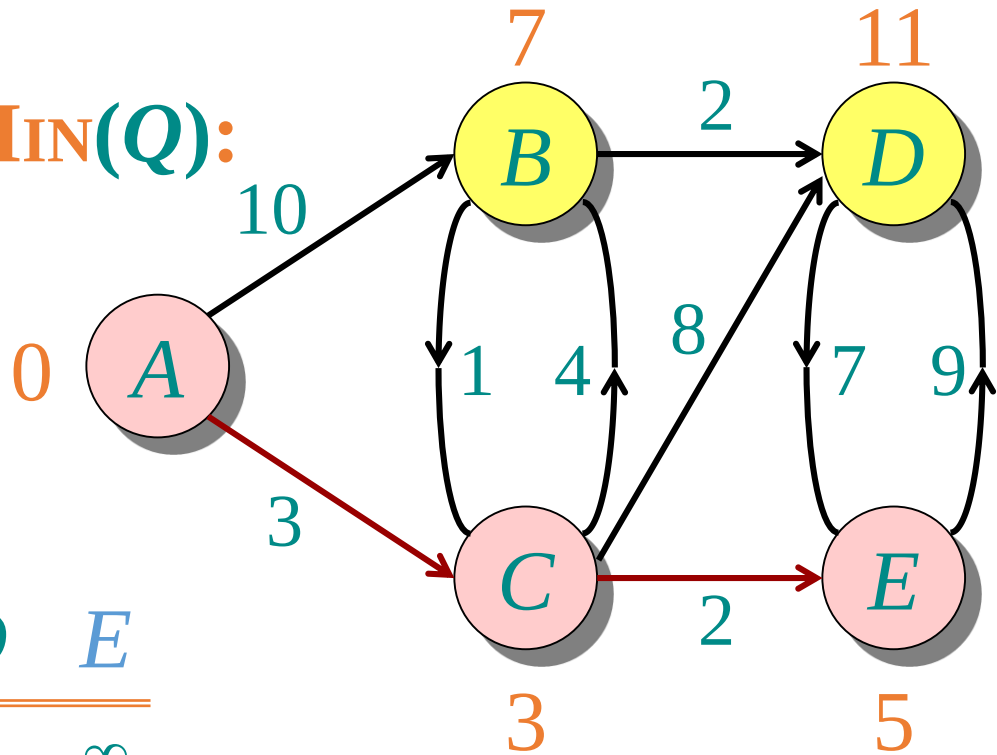


Example of Dijkstra's algorithm

"E" ← EXTRACT-MIN(Q):

$K: \{ A, C, E \}$

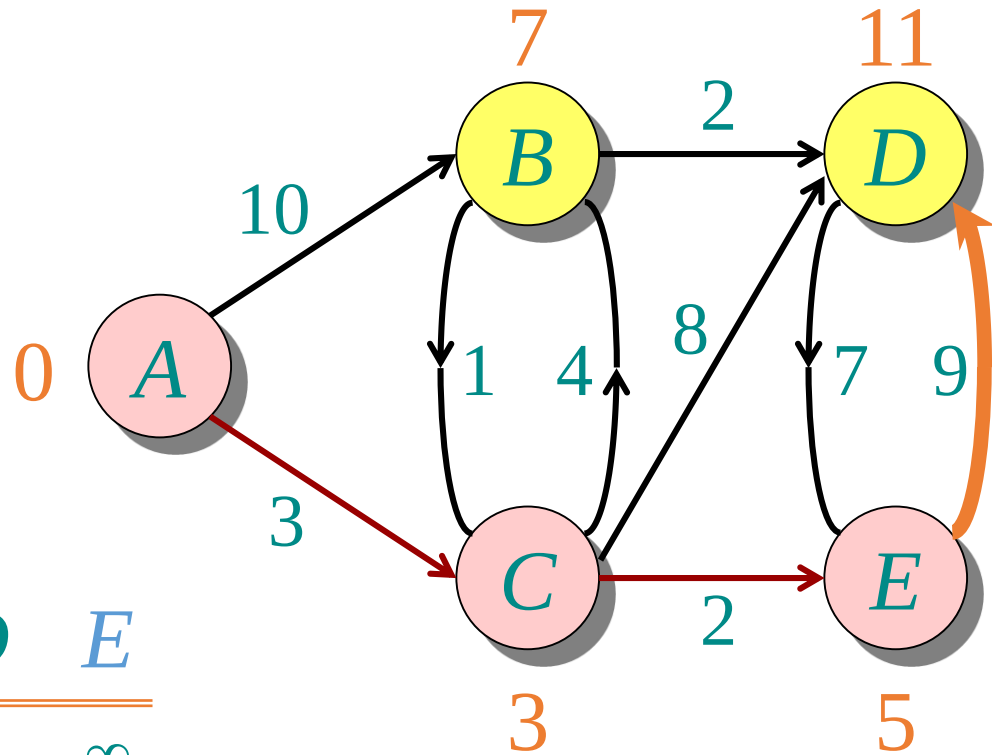
Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	—	—
		7		11	5



Example of Dijkstra's algorithm

Relax all edges
leaving *E*:

$K: \{ A, C, E \}$



Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5
		7		11	

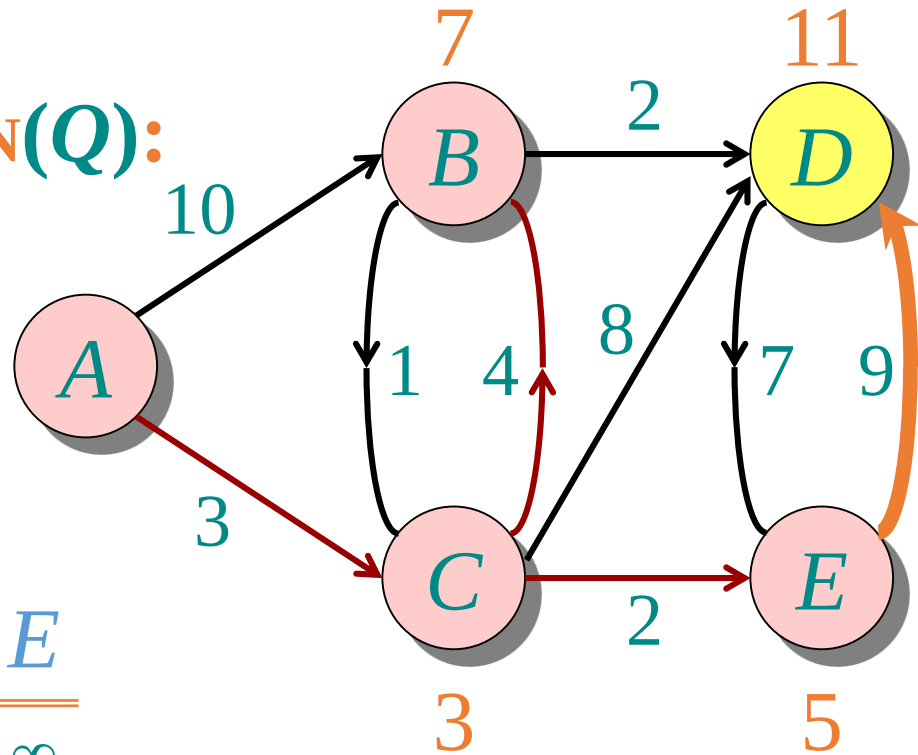
Example of Dijkstra's algorithm

"B" $\leftarrow$ **EXTRACT-MIN(Q):**

$K: \{ A, C, E, B \}$

0

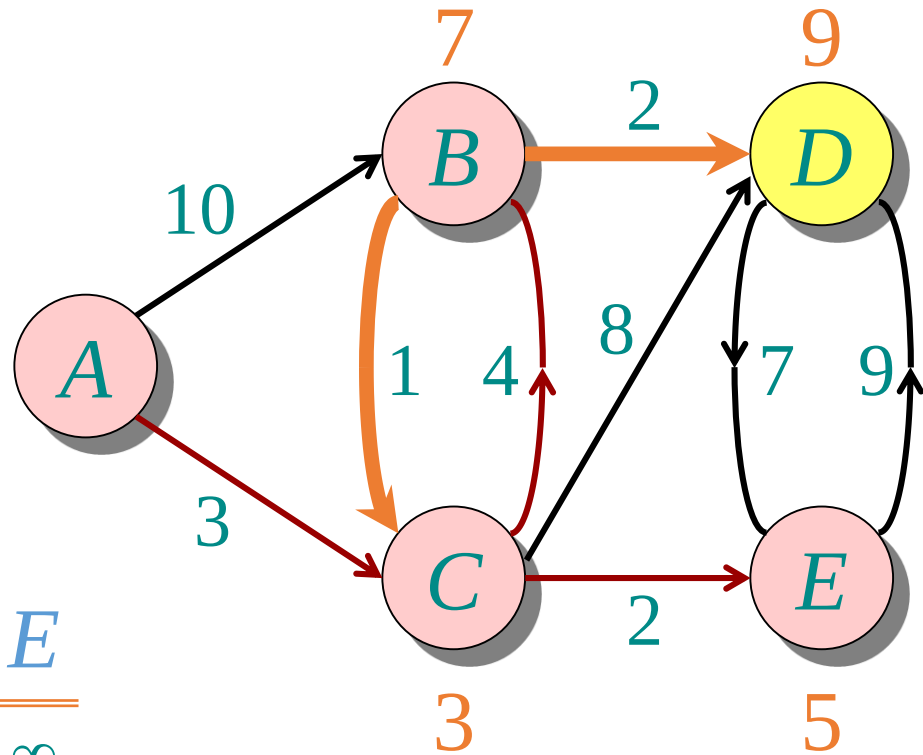
Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5
		7		11	



Example of Dijkstra's algorithm

Relax all edges
leaving **B**:

$K: \{ A, C, E, B \}$ 0

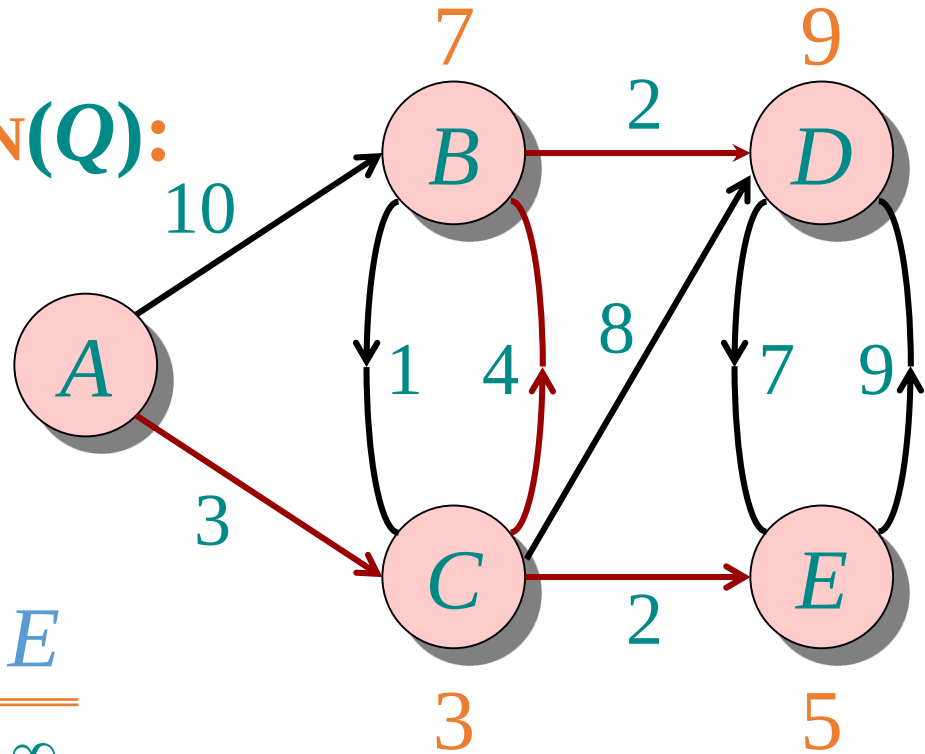


Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5
		7		11	
				9	

Example of Dijkstra's algorithm

"D" $\leftarrow$ **EXTRACT-MIN(Q):**

K: { A, C, E, B, D }



Q:	A	B	C	D	E
	0	∞	∞	∞	∞
		10	3	∞	∞
		7		11	5
		7		11	
				9	

Dijkstra (G, w, s) – Time Complexity

1. INITIALIZE-SINGLE-SOURCE(V, s) $\leftarrow \Theta(V)$
2. $K \leftarrow \emptyset$
3. $Q \leftarrow V[G] \leftarrow \Theta(V)$ time to build min-heap
4. **while** $Q \neq \emptyset \leftarrow$ Executed $\Theta(V)$ times
5. **do** $u \leftarrow \text{EXTRACT-MIN}(Q) \leftarrow V$ times, $\Theta(T_E)$ each time
6. $K \leftarrow K \cup \{u\}$
7. **for** each vertex $v \in \text{Adj}[u]$
8. **do** $\text{RELAX}(u, v, w) \leftarrow \Theta(E)$ times, $\Theta(T_D)$ each time

Total running time: $\Theta(V + V \times T_E + E \times T_D)$

T_E = time taken by an EXTRACT-MIN operation

T_D = time taken by a DECREASE-KEY operation

Dijkstra's Time complexity (cont.)

1. Priority queue is an array.

EXTRACT-MIN in $\Theta(V)$ time, DECREASE-KEY in $\Theta(1)$

Total time: $\Theta(V + VV + E) = \Theta(V^2)$

2. ("Modified Dijkstra")

Priority queue is a binary (standard) heap.

EXTRACT-MIN and DECREASE-KEY takes $\Theta(\lg V)$ time each

Total time: $\Theta(V + V\lg V + E\lg V) = \Theta(E\lg V)$